



MLOps: Fast API

Predicción de Abandono de Clientes Bancarios

Rodrigo De La Fuente y Susana Alvarez

MLOps | MIOTI



Contenido

01 Descripción General

- Introducción al proyecto y objetivo principal.

02 Dataset y Entrenamiento

- Selección y Exploración del Dataset
- Preprocesamiento de Datos

03 Registro, Comparación y Selección

- Configuración y uso de MLFlow
- Registro de parámetros, métricas y artefactos

04 Despliegue y Validación

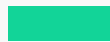
- Validación de datos, logging y pruebas automatizadas con Fast-API
- 



1. Descripción General

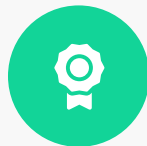
Introducción al proyecto y objetivo principal

Introducción



Descripción del proyecto

El proyecto implementa un pipeline completo de **MLOps** que combina el entrenamiento y registro de modelos con **MLflow** y la exposición de predicciones en tiempo real mediante una **API REST** construida con **FastAPI**.



Objetivo general

Predecir si un cliente bancario nacional abandonará la institución, a partir de sus características personales y financieras.



2. Dataset y Entrenamiento

Selección y exploración del dataset, preprocesamiento de datos y división para entrenamiento

Problema y Dataset

El objetivo es predecir la **deserción de clientes** (churn) en un banco, permitiendo anticipar bajas y tomar acciones preventivas.



Datos

+10,000 clientes, con variables demográficas*, financieras y de comportamiento.



Variable objetivo

Exited

- 1 = abandonó
- 0 = sigue

*Filtrado por datos nacionales.



Variables

target	VARIABLE	DESCRIPCIÓN
	credit_score	Puntuación crediticia
	gender	Género (Male / Female)
	age	Edad
	tenure	Años como cliente
	balance	Saldo en cuenta
	product_number	Número de productos contratados
	credit_card	Tiene tarjeta de crédito (0/1)
	active_member	Es miembro activo (0/1)
	estimated_salary	Salario estimado
	churn	Abandona (0/1)

Pipeline de Entrenamiento

01

02

03

04

Preprocesamiento

Se filtra por país, se codifica el género y se eliminan columnas irrelevantes. Se aplica `StandardScaler` a las variables numéricas continuas.

Train/Test Split

80% entrenamiento / 20% prueba, con `random_state=42`.

Entrenamiento

Se entrenan los modelos y se almacenan en mlruns con su estructura de artefactos.

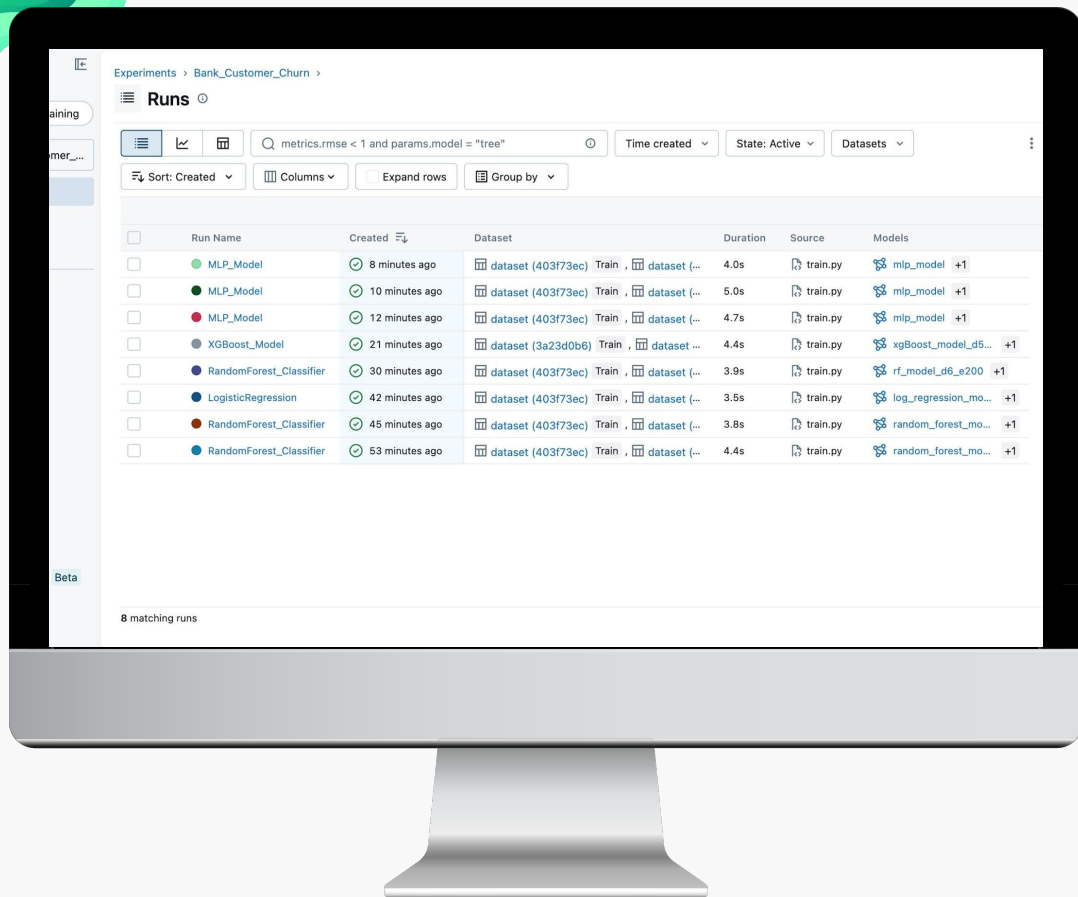
MLflow Logging

Registro y Seguimiento con MLFlow para el ajuste de hiperparámetros, exportación y registro del **Mejor Modelo**



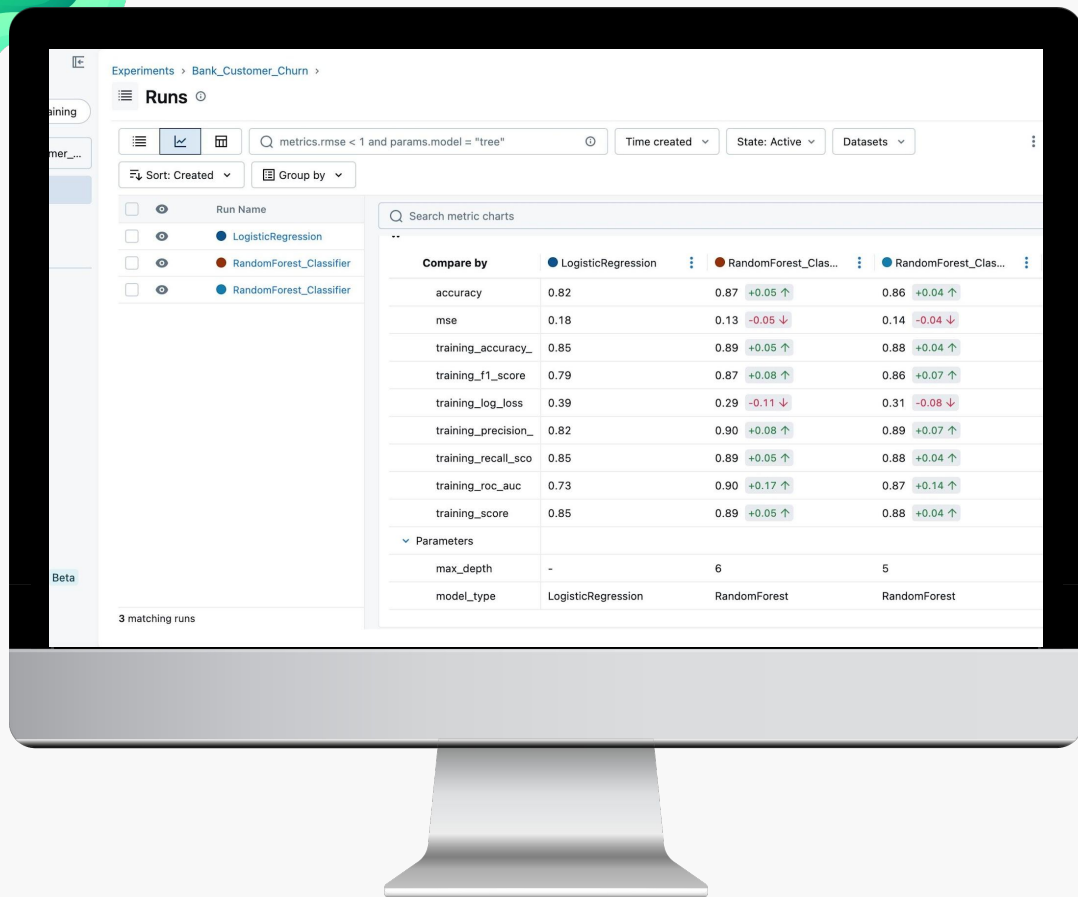
3. Registro, Comparación y Selección

Registro y Seguimiento con MLFlow, ajuste de
hiperparámetros y exportación y registro del Mejor
Modelo



Registro y Seguimiento con MLFlows

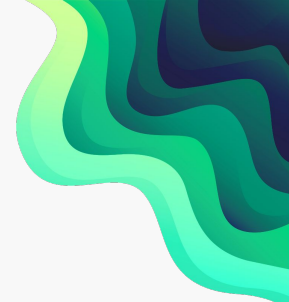
- MLflow actúa como el **registry** y **servidor de tracking** del proyecto.
- Los modelos entrenados se almacenan en *mlruns* con su estructura de artefactos (MLmodel, conda.yaml, requirements.txt).
- La ruta al modelo en uso se configura via variable de entorno `MLFLOW_MODEL_PATH` en el archivo `.env`.
- La UI de MLflow se levanta con `mlflow ui` para comparar experimentos y versiones.



Exportación y Registro del Mejor Modelo

Segun la experimentación realizada, el mejor resultado registrado fue `XGBClassifier`` (XGBoost) con parámetros:

- ``n_estimators = 100``
- ``max_depth = 5``
- ``learning_rate = 0.1``.



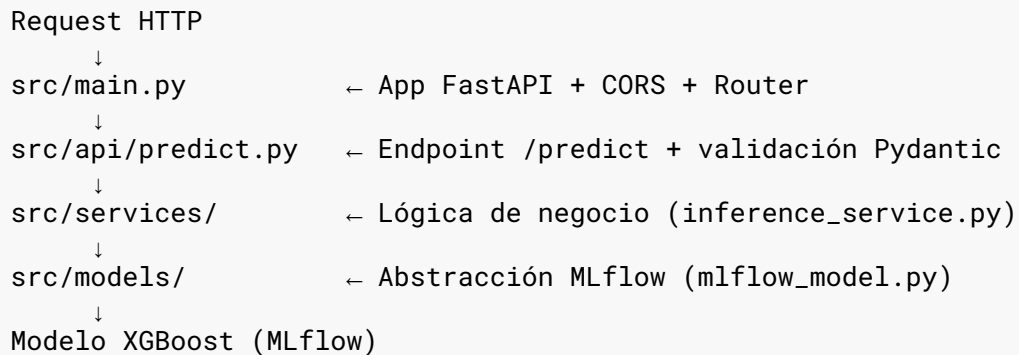
4. Despliegue y Validación

Despliegue con FastAPI y Pruebas del Endpoint



Arquitectura de la API — FastAPI

La aplicación sigue una **arquitectura en capas**:



Despliegue con FastAPI y Pruebas del Endpoint

Endpoints

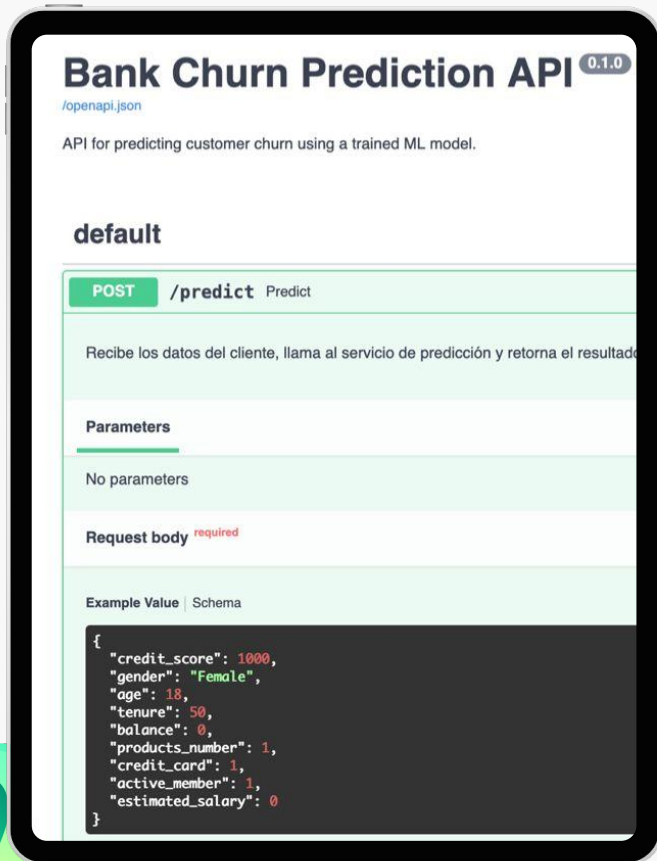
Método | Ruta | Descripción

GET	/health	Verificación de disponibilidad
POST	/predict	Predicción para un cliente

Validación de Entrada (Pydantic)

El esquema CustomerData valida automáticamente:

- credit_score: entero entre 0 y 1000
- gender: solo acepta "Male" o "Female"
- age: entre 18 y 120
- balance, estimated_salary: valores no negativos



Configuración Centralizada — `config.py`

Se usa pydantic-settings para gestionar la configuración mediante variables de entorno o archivo `.env`:

Variable	Valor por defecto
API_HOST	0.0.0.0
API_PORT	8000
ENVIRONMENT	development
LOG_LEVEL	INFO
MLFLOW_MODEL_PATH	Ruta al artefacto del modelo

Testing — test_api.py

Se implementan **3**
pruebas con pytest y
TestClient de FastAPI:

Ejecutar con:
bash
pytest

Test

test_health_check | Verifica que /health retorna {"status": "ok"}

test_predict_valid | Verifica predicción exitosa con datos válidos

test_predict_invalid | Verifica rechazo HTTP 422 con datos inválidos

| Validación

```
(mlops-fastapi) rodrigo@MacBook-Air-de-Rodrigo MLOps-Fast-API % curl http://127.0.0.1:8000/health  
{"status": "ok"}  
-
```



Logging — `logger.py`

Logger centralizado utilizado en todas las capas de la aplicación para trazabilidad de predicciones y errores en tiempo real.

Flujo Completo de Uso

```
bash
# 1. Instalar dependencias
pip install -r requirements.txt

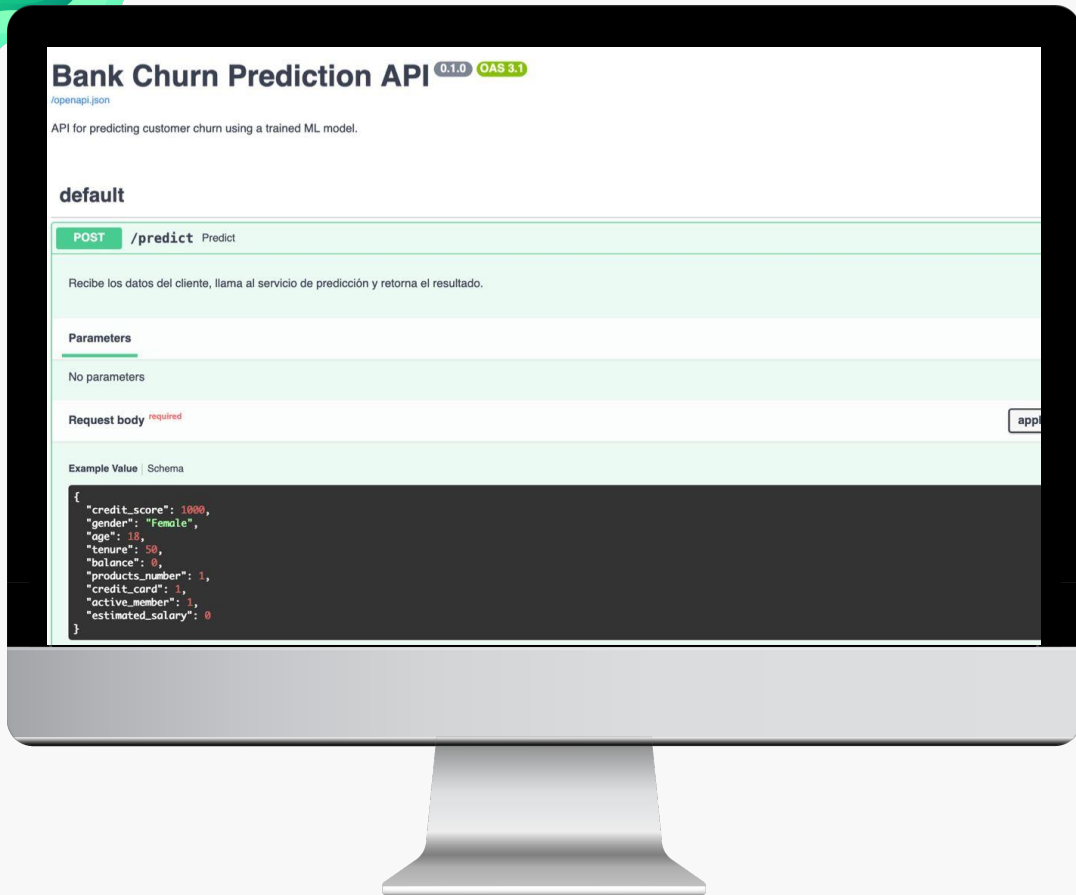
# 2. Descargar datos y entrenar
python src/load_data.py
python src/train.py

# 3. Verificar y copiar ruta del modelo en .env
ls -td mlruns/*/models/*/artifacts | head -n 1

# 4. Levantar MLflow UI (opcional, para explorar experimentos)
mlflow ui

# 5. Levantar la API
uvicorn src.main:app --reload

# 6. Consultar la API
curl http://127.0.0.1:8000/health
curl -X POST http://127.0.0.1:8000/predict -H "Content-Type:
application/json" \
  -d '{"credit_score": 650, "gender": "Male", "age": 35, ...}'
```



Levantar la API

uvicorn src.main:app --reload

default

POST /predict Predict

GET /health Health Check

Parameters

No parameters

Responses

Code	Description
200	Successful Response

Media type

application/json

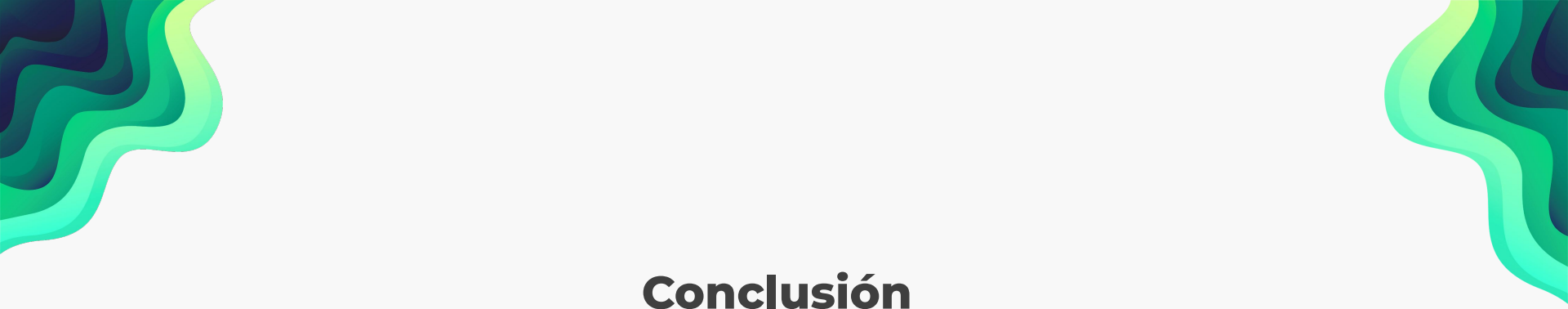
Controls Accept header.

Example Value | Schema

"string"

Consultar la API

```
(mlops-fastapi) rodrigo@MacBook-Air-de-Rodrigo MLOps-Fast-API % curl -X POST http://127.0.0.1:8000/predict \
-H "Content-Type: application/json" \
-d '{
  "credit_score": 650,
  "gender": "Male",
  "age": 35,
  "tenure": 5,
  "balance": 50000.0,
  "products_number": 2,
  "credit_card": 1,
  "active_member": 1,
  "estimated_salary": 60000.0
}'
{"churn_prediction":0}
```



Conclusión

El proyecto implementa una solución MLOps completa para la predicción de abandono de clientes bancarios, integrando entrenamiento, registro y despliegue de modelos de Machine Learning mediante XGBoost, MLflow y FastAPI.

La arquitectura en capas permite separar la lógica de inferencia, configuración y acceso al modelo, mejorando la mantenibilidad y escalabilidad del sistema.

Además, la incorporación de validación de datos, logging y pruebas automatizadas garantiza un servicio robusto y preparado para evolucionar hacia entornos de producción.

Gracias